

SpatialOS

Shared Spatial Operations and Localization Reliability

Milestone 1

User Stories and High Level Requirements

Section 06 | Team 02

September 29, 2026

Team member	M0 role
Daksh Kumar	Team Lead
Sumit Kumar	AI Master
Calvin Mei	Front-end Lead
Noah Bran	Back-end Lead
Tyler Duong	Scrum Master
Ezeren Masangcay	GitHub Master

GitHub repository

<https://github.com/ezerenm/SFSU-csc648-06-Fall26-team02>

Revision history

Version	Date	Revision summary
v1.0	Sep 29, 2026	Initial M1 submission

1 Executive Summary

When people and robots share indoor spaces, operators need consistent path information and evidence that positioning is reliable. In our demonstration, a person and a simulated robot both need to reach Room 103. Hallway B is open to people but restricted for robots, so each receives an appropriate route. A scripted blockage report from the simulated robot goes to an operator for confirmation. Once confirmed, every affected view receives the same closure and updates its route. The operator then checks how reliably the phone located itself at the tested positions.

SpatialOS is a shared operations and reliability layer for spaces used by people and robots. Our semester version connects one scanned campus area, a real phone, and a simulated robot through shared path information and repeatable localization tests. Campus technology teams and spatial developers can inspect route behavior and positioning evidence in the same application before expanding a deployment.

Product experience

A participant points the phone camera around the mapped area. The web viewer displays a valid position and a route to a selected destination. Human and simulated-agent routes use the same graph, with robot access rules applied where needed. Operators manage closures, and the application reports an unreachable destination when no permitted route exists.

The dashboard records successful and failed positioning attempts. Green, yellow, red, and gray test-point markers summarize measured localization health and data sufficiency. Gemini explains selected results in plain English, links them to the supporting measurements, and separates observations from suggested follow-up tests.

Semester commitment

The MVP covers one approved area, one map version, one phone platform, three destinations, five test points, and manually defined paths. Core features are live positioning, shared routes and closures, robot access rules, simulation, and localization health. Mid-priority features add replay, exports, Gemini summaries, and operator-reviewed agent reports. Simulation remains clearly labeled.

An eight-week implementation plan covers M2 design, M3 prototype, and M4 beta readiness, followed by M5 preparation for early December. Physical robots, automatic obstacle or stale-map detection, campus-wide mapping, and advanced AR guidance remain future extensions.

Value and commercial direction

The commercial direction is a B2B subscription per managed site, with optional capture and onboarding services. Interviews and observed pilot tasks will test the value of the combined workflow and guide pricing.

Our six-person team will build on its existing web foundation with named owners and backup coverage for mobile integration, AI, routing, rendering, and testing. SpatialOS turns a captured space into shared operations that teams can demonstrate, measure, and improve.

2 Personas and User Stories

The following initial personas guide requirements. Planned interviews and usability sessions will refine them.

P1 Campus visitor. Uses a phone, has little knowledge of the building, and wants a clear route to a room. Confusing location markers and unavailable paths undermine confidence.

P2 Spatial developer. Can interpret logs and test results. Needs repeatable positioning trials, failure records, and map-version comparisons rather than relying on a convincing demonstration.

P3 Site operator. Knows which passages are available and maintains destination information. Needs controlled updates that reach every active navigation view.

P4 Simulation tester. Understands basic navigation graphs and wants to check how an agent responds to destinations and closures without operating a physical robot.

Navigation stories

US1 | High | As a visitor, I want to see my current position and its status so I can decide whether it is usable.

US2 | High | As a visitor, I want to choose a destination and see a route so I can find it.

US3 | High | As a visitor, I want a revised route or an unreachable message when a passage closes so I know what changed.

US4 | High | As a visitor, I want an unavailable or stale-location message so I do not mistake an old position for my current location.

Operations and simulation stories

US5 | High | As an operator, I want to maintain approved destinations and paths so the map reflects the pilot area.

US6 | High | As an operator, I want to set passage availability and robot access, with history, so each user receives a permitted route.

US7 | High | As a simulation tester, I want an agent to follow permitted paths and stop or replan after changes so I can verify shared navigation.

US8 | Mid | As a simulation tester, I want to select a starting point or replay a saved session so I can repeat a demonstration without live positioning.

US13 | Mid | As an operator, I want to confirm or reject a simulated-agent blockage report so shared closures reflect reviewed information.

Evaluation stories

US9 | High | As a developer, I want every localization attempt recorded with its context so failures remain visible.

US10 | High | As a developer, I want health status, sample counts and measured results at each test point so I can identify locations needing investigation.

US11 | Mid | As a developer, I want to filter and export trial records so another team member can reproduce the analysis.

US12 | Mid | As an operator, I want a plain-English summary of selected trial results so I can understand performance and identify follow-up tests.

3 Data Definitions

Entity or term	Meaning and main logical information
User and role	Identity, role and site access. Visitor: navigate and view own session. Developer/tester: run trials and simulate. Operator: edit paths and review reports. Administrator: manage accounts and site configuration.
Site and map version	Site identifies the physical pilot area. A map version identifies its captured assets, publication state, date, and alignment. Each graph and session references a specific version.
Visual asset and VPS map	The visual asset is the displayed mesh or splat. The VPS map supports camera-based localization. They serve different purposes and must be aligned; neither automatically defines safe routes.
World frame and transform	The common origin, axes, orientation convention, and meter scale. A versioned transform converts a returned device pose or visual asset into this frame.
Navigation graph	Versioned nodes and path segments (edges). Each segment has endpoints, direction, distance, availability (open/closed), and robot access (allowed/restricted). Reopening preserves its robot access rule.
Destination	A named place with a description, site, map version, and graph node. The pilot contains three destinations.
Path event and report	Events record availability/access changes, actor, time and revision. A blockage report records segment, agent, time, reason and pending/confirmed/rejected status. Confirmation links it to one closure event.
Session and pose sample	A session identifies user, device, map version, and live/simulation/replay mode. A pose sample records position, orientation, capture time, tracking status, and receipt time.
Route and simulated agent	A route records start, destination, ordered segments, distance, human/robot profile, mode and path-state revision. An agent records progress and running/paused/arrived/unreachable state.
Test point and reference	A marked physical camera position with identifier, measured coordinates, reference method, uncertainty, and alignment version. Points used for alignment are separate from evaluation points.
Localization attempt	Trial ID, start/end time, outcome, device, SDK/map version, point, condition, camera direction, and optional valid pose/reference. Failures include timeout, tracking loss, network error and frame mismatch.
Metric summary and health	Sample counts, success rate, successful-attempt latency and valid horizontal error for a selected cohort. Health is red/yellow/green/gray under Section 5 rules; it describes tested positioning, not automatic map freshness.
AI summary	Generated explanation linked to a metric snapshot and trial selection, with model/prompt version and generation time. It cites measured results and keeps suggestions separate from observations.

A site has map versions; each version links a graph, destinations, sessions and test points. Sessions contain pose samples and attempts. Routes reference graph segments and a path-state revision. Reports target segments; operator confirmation updates shared state.

Positioning success means obtaining a valid pose for the intended map and frame within 30 seconds. Spatial accuracy is evaluated separately. A pose older than five seconds, or with invalid tracking, is not a current navigation start.

4 Initial Functional Requirements

High requirements define the core; Mid features follow it. IDs remain stable for later milestones. Section 8 identifies owners.

ID and title	Priority and story	Required behavior
R1 Access control	High US5, US6, US9	Authenticate participants and enforce the role and site permissions defined in Section 3.
R2 Shared environment	High US1, US5, US7	Display the approved site asset, three destinations, and versioned path network in one spatial frame.
R3 Live localization	High US1, US9	Request real phone localization and deliver its pose, map identity, status, and timestamps to the application.
R4 Position status	High US1, US4	Show pose age and tracking state. Mark stale/lost positions and reacquire before a new live route.
R5 Destination routing	High US2	Find a shortest permitted open route for the human or robot profile; show its segments and distance.
R6 Start validation	High US2, US4	Attach starts only to nearby verified walkable segments; explain off-network or map-mismatch errors.
R7 Map administration	High US5	Let operators edit destinations and graph records, validate references, and publish a consistent revision.
R8 Shared closures	High US3, US6	Record authorized edge closures or reopenings and deliver the committed revision to active viewers.
R9 Route revision	High US3, US7	Invalidate affected routes after availability/access changes; replan or show unreachable.
R10 Agent simulation	High US7	Start, pause and reset one simulated agent. Follow permitted segments; stop or replan after changes.
R11 Simulation and replay	Mid US8	Support a manually selected start and recorded pose replay with persistent mode labels and separate evaluation filters.
R12 Trial logging	High US9	Persist a unique record for every initiated evaluation attempt, including failure, timeout, condition, and optional pose/reference.
R13 Evaluation dashboard	High US10	Show trial counts, success, latency, failures and valid position error, with cohort filters and sample counts.
R14 Localization health	High US10	Color tested points using Section 5 rules; show counts, conditions and error coverage. Mark insufficient data gray.
R15 Filter and export	Mid US11	Filter trials by point, device, condition, version, outcome, and mode; export the matching records for reproducible analysis.
R16 AI health summary	Mid US12	Use Gemini to explain selected health results with supporting metrics and trial references. Preserve the dashboard if generation fails.
R17 Robot access	High US6, US7	Let operators restrict robot access per segment. Apply it only to robot routes and preserve it across closures.
R18 Agent reports	Mid US13	Pause the agent on a scripted blockage report. Operators confirm/reject it; confirmation alone closes the segment globally.

5 Nonfunctional Requirements

Acceptance testing will use the targets below and retain the test conditions and results.

ID	Quality and measurable target
N1	Capacity and response: support 20 concurrent web sessions, one phone, one simulated agent and up to 100 nodes/200 edges. At least 95 of 100 route requests finish within one second, excluding VPS time.
N2	Rendering: at 20 Mbps on the recorded test laptop, the pilot scene becomes interactive within 10 seconds in 9 of 10 cold loads. Maintain 30 FPS during a 60-second demo; record hardware and asset size.
N3	Consistency: 95% of 30 availability/access changes update both views within two seconds of commit. Reject stale writes; confirming the same report twice creates one event.
N4	Failure handling: time out attempts without a valid pose after 30 seconds; mark poses over five seconds old as stale. Queue failed telemetry, deduplicate retries by attempt ID and disclose unrecovered attempts.
N5	Security and privacy: enforce server and database role checks for all writes; pass unauthorized-access tests for every protected operation. Keep secrets out of Git and browser bundles. Do not retain raw camera recordings in the application by default.
N6	Usability: test the browsers listed in Section 7. Non-3D controls support keyboard use and text status. At least 4 of 5 pilot users complete a destination task without coaching.
N7	Storage: budget 1 GB for 10,000 attempts and 100,000 poses, plus 1 GB for spatial assets. Export daily during field tests and demonstrate a successful restore.
N8	Availability: pass 99% of application health checks over a seven-day evaluation, sampled every five minutes. Report third-party localization availability separately and show useful failure states.
N9	Maintainability: merged features require teammate review, lint, type checks, and relevant tests. Pin packages and document setup, frames, APIs, graph fixtures, and experiments.
N10	AI reliability: test 20 fixed snapshots, including empty/failed-only data. Numbers must match their source; reject unsupported claims. Label AI text and preserve the dashboard after a 15-second timeout.

Evaluation method

Run 50 live trials: five points, two recorded conditions, five repeats on one device/map version. Vary order; retain failures and direction labels. Report success/all attempts and median/p95 successful latency with counts. Survey fixed camera positions independently, using separate alignment and evaluation points; record camera offset and uncertainty. Target 80% success within 30 seconds and median horizontal error at most 1 m.

Test point health rules

Use live trials from the same test point, device and map version; median error means horizontal error.

Status	Condition
Gray	Fewer than 10 attempts; or at least 10 attempts with success at least 60% but fewer than five valid error samples.
Red	At least 10 attempts, and either success below 60% or median error above 1.5 m with at least five valid error samples.
Green	At least 10 attempts and five valid error samples, success at least 80%, and median error at most 1 m.
Yellow	All remaining cases with at least 10 attempts and five valid error samples.

Show counts and conditions with each label. Five-attempt condition subsets stay gray. Untested space is unknown; these thresholds do not establish map freshness.

6 Competitive Analysis

SpatialOS combines capabilities from indoor mapping, localization, and navigation testing. Niantic Spatial also supplies the localization foundation used by our application.

Product	Phone VPS	Human wayfinding	Robot routes or operations	Localization data review
Niantic Spatial [1,2]	✓	✓ sample	✗	✓
Mappedin [3,4]	✓	✓	✗	✗
Nav2 [5]	✗	✗	✓	✓
Formant [6]	✗	✗	✓	✓
InOrbit [7]	✗	✗	✓	✓
SpatialOS plan	✓	✓	✓ simulated	✓

✓ = documented capability; ✗ = not documented as a packaged feature in the cited sources. SpatialOS checks identify planned features.

SpatialOS focuses on a repeatable site pilot: phone localization trials, shared human/simulated-agent paths, robot restrictions, operator-confirmed reports and readable health results. Formant and InOrbit already expose localization and spatial operations data. Our advantage is the focused combination of a fixed-point experiment protocol with shared route behavior and an AI explanation in one campus demonstration.

Validation and adoption

Tyler will coordinate interviews with three potential users/developers and observe five campus participants. Tasks cover finding a destination, interpreting a health marker, and understanding a confirmed closure. Daksh and Sumit will prioritize improvements from the findings.

Sources for product and platform facts

Official documentation consulted September 29, 2026.

[1] Niantic Spatial SDK overview. <https://www.nianticspatial.com/docs/nsdk/>

[2] Niantic Spatial release notes and wayfinding sample. https://www.nianticspatial.com/docs/nsdk/release_notes/index.html

[3] Mappedin developer overview. <https://developer.mappedin.com/docs/overview>

[4] Mappedin indoor positioning. <https://developer.mappedin.com/docs/enterprise-apps/mappedin-web/indoor-positioning>

[5] Nav2 route server. https://docs.nav2.org/jazzy/configuration_and_development/configuration_guide/core_servers/route_server/configuring_route_server/

[6] Formant localization streams. <https://docs.formant.io/docs/getting-started-add-a-ros-localization-stream>

[7] InOrbit maps, localization and operations. <https://www.inorbit.ai/docs>

7 High Level System Requirements

Retain the team's submitted M0 web stack. Add spatial rendering, a native localization client, and the testing tools listed below. Resolve exact compatible spatial versions during the initial integration study and commit their versions before feature integration.

Component	Proposed framework or service
Web application	Next.js 16 App Router, React 19, Tailwind CSS, and TypeScript; Node.js 22. Next.js server/API routes provide application endpoints.
Database and identity	Supabase PostgreSQL 17 and Supabase Auth, with row-level security. Supabase Storage holds approved visual assets; signed access protects private captures.
Capture and localization	Niantic Scaniverse [8], a generated active VPS map, and Niantic Spatial SDK. Confirm student Pro organization access through the course TA and validate credits and device support.
Phone client addition	Proposed Unity/C# client using Niantic Spatial SDK on one supported Android phone. It localizes and sends authenticated, timestamped poses to our API. Device access and build compatibility are an initial checkpoint; browser-based VPS is not assumed.
Spatial viewer addition	Three.js with its compatible asset loader; use an optimized mesh first. Blender may convert or simplify the exported mesh while preserving documented scale and transforms.
Routes and state	Team-written Dijkstra routing with human/robot profiles. Supabase Realtime distributes path-state revisions with polling recovery. Operator confirmation commits agent reports; pose uploads target 2 Hz.
Testing and delivery	GitHub, GitHub Actions, Visual Studio Code, ESLint, TypeScript checks, Vitest, and Playwright. Vercel hosts the web app and API. Native client installs separately on the test phone.
AI results summary	Google Gemini API via the Google GenAI SDK (@google/genai), called from a protected server endpoint. Structured output [9] summarizes server-calculated metrics; route planning and arithmetic remain deterministic.
Supported browsers	Current stable Chrome and Firefox on desktop, Chrome on Android, and Safari on iOS. Record tested versions at beta. The Unity Android client handles live camera localization.

Integration contract

The phone sends session ID, unique sample/attempt ID, site and map version, position, orientation, status, and timestamps over HTTPS. The backend validates identity and frame compatibility, stores trials, and serves the current pose to the web viewer. Routes reference the same world frame and path-state revision. Keep raw provider poses alongside the transform version so alignment errors can be reproduced.

Validate live integration at three points; publish the VPS asset version to Production [8]. R11 supports manual starts and recorded replay. Gemini receives aggregate metrics and pseudonymous trial references. Health rules run in application code; Gemini explains results and suggests follow-up checks without diagnosing stale maps or changing routes.

[8] Niantic Scaniverse quickstart and map publication workflow. <https://www.nianticspatial.com/docs/scaniverse/quickstart/>

[9] Google Gemini API structured outputs. <https://ai.google.dev/gemini-api/docs/structured-output>

8 Team and Study Plan

Each member owns a deliverable and participates in review and testing. M0 roles remain unchanged. Cross-training protects the mobile integration and frontend work from single-person dependencies.

Member and role	Feature ownership and support
Daksh Kumar Team Lead	Phone localization, scan/VPS preparation, coordinate alignment, architecture and mobile setup documentation; R3-R4, R6. Sumit is the Unity/client backup.
Sumit Kumar AI Master	Routing, basic simulation, replay, health calculations and Gemini validation; R5, R9-R11, R13-R14, R16-R17. Reproduces the Unity build as backup; Tyler leads trial execution.
Noah Bran Back-end Lead	Database, authentication, APIs, telemetry, shared state and report confirmation; R1, R7-R8, R12, R18. Supports access rules and the Gemini endpoint.
Calvin Mei Front-end Lead	Dashboard layout, filters, controls, status labels and API-result display; UI for R4, R13-R16, R18. Ezeren reviews viewer integration and supports frontend work.
Ezeren Masangcay GitHub Master	Three.js viewer, model loading, pose/route rendering, integration, CI, deployment and export; R2, R3-R5, R15. Supports replay integration and Calvin's dashboard.
Tyler Duong Scrum Master	Test cases, localization trials, usability sessions, results collection and reproducible bug reports. Coordinates twice-weekly meetings, minutes and acceptance evidence for R1-R18.

Technology study within the first month

Study targets run from September 29 through October 13, 2026. Each owner records the exercise result and shares a short walkthrough with the team.

Study lead and participants	Exercise and evidence	Target
Daksh; Sumit	Confirm Niantic access and phone setup by Oct 2. Study Scaniverse, SDK/Unity and transforms; record live poses at three points by Oct 7. Daksh documents setup; Sumit pairs on the build.	Oct 2 setup Oct 7 poses
Ezeren; Calvin and Daksh	Ezeren loads the mesh, pose and route in Three.js and measures FPS. Calvin connects dashboard controls and status labels; Daksh verifies frames.	Oct 6
Noah; Ezeren and Tyler	Study Supabase Auth/RLS, Storage, Realtime and Next.js APIs. Demonstrate protected writes, trial insertion and two-client closure updates.	Oct 6
Sumit; Noah and Calvin	Study Gemini structured output. Build the protected summary endpoint, display cited metrics, and test empty/failure-only input.	Oct 9
Sumit and Tyler; Daksh	Sumit checks routing and health calculations. Tyler writes test cases, runs trials and records results; Daksh supports the reference survey.	Oct 9
Ezeren and Tyler; whole team	Practice Git reviews, CI, Vitest/Playwright and field procedures. Each member submits a reviewed change; archive an integrated test report.	Oct 13

Daksh will document the Unity phone setup, and Sumit will independently build and run the client by October 9 to confirm backup readiness.

Delivery timeline

Team delivery targets below align the build with M2-M5. Canvas announcements govern official submission dates; early December is the planned Demo Day window.

Build period	Milestone alignment and deliverable
Weeks 1-2 Sep 29-Oct 12	M2 design readiness: architecture, UI mockups, data/API contracts, scan and initial localization integration.
Weeks 3-4 Oct 13-26	M3 prototype readiness: live start, profile-based routes, simulation, shared closures, trials and first Gemini summary.
Weeks 5-6 Oct 27-Nov 9	M4 beta readiness: confirmed agent reports, 50 live trials, health markers, usability/AI validation and QA results.
Weeks 7-8 Nov 10-23	M4 stabilization and M5 preparation: regression tests, backup-client handoff, replay fallback, documentation and demo rehearsal.
Nov 24-early Dec	M5 final delivery: resolve final feedback, freeze the release, submit the final package and demonstrate SpatialOS at Demo Day.

9 Checklist

Required checklist item	Status	Evidence or next step
Outside-class meeting time agreed	DONE	The team meets twice a week, including its outside-class meeting.
Scrum Master shares meeting minutes	DONE	Tyler maintains and shares the team meeting document.
GitHub master chosen	DONE	M0 names Ezeren Masangcay as GitHub Master.
Local development environment	ON TRACK	Complete all local setups and add the phone-client build instructions.
Team agreement on tools and deployment	DONE	All six members agreed to the listed tools, SpatialOS additions and deployment plan.
Frontend and backend framework readiness	ON TRACK	Continue the assigned integration exercises and shared walkthroughs.
Technology study owners and goals	DONE	Section 8 assigns participants, exercises and calendar deadlines.
Everyone read and agreed to final M1	DONE	All six members reviewed and approved the final M1 document.